Antigravity High-Performance Typesetting & Architecture Report

A Publication-Grade PDF Generation Pipeline Powered by Headless Chromium and Modern CSS Paged Media

Abstract: This report presents the architectural design, implementation details, and typesetting optimization pipeline of the Markdown-to-PDF skill for Google Antigravity. By pairing native Markdown semantics with KaTeX vector math rendering, Pygments code syntax styling, and headless Chromium print automation, the engine delivers publication-grade PDF outputs alongside real-time interactive preview artifacts.

Author: haownz

Date: 2026-10-11

Version: v1.0.0

Table of Contents

1. Executive Summary & Design Principles
2. System Architecture
3. Mathematical Formula Typesetting
4. Code Block Syntax Highlighting
5. Core Feature Matrix
6. Conclusion

1. Executive Summary & Design Principles

In agentic programming and AI-assisted engineering workflows, exporting complex analytical write-ups, technical architecture specifications, and empirical benchmarks into **publication-grade PDF documents** is a frequent requirement. Conventional conversion toolchains often suffer from several critical shortcomings:

1. **Suboptimal Typography**: Inconsistent font hierarchies, poor fallback choices, and rigid text wrapping.
2. **Fragile Math & Diagram Rendering**: Loss of vector clarity or missing font glyphs when rendering LaTeX equations or architecture charts.
3. **Uncontrolled Page Breaks**: Orphaned headings (`break-after: avoid`) left hanging at the bottom of pages, and table rows sliced awkwardly across page boundaries.
4. **Disconnection from Agent Workspaces**: Inability to immediately inspect generated documents within the host workbench.

> **NOTE**
>
> The `markdown-to-pdf` skill resolves these pain points. By decoupling typesetting styles from the underlying browser engine, it produces crisp, publication-grade PDFs while automatically emitting Generative UI artifacts for seamless Antigravity inspection.

---

2. System Architecture

The end-to-end pipeline is structured into four deterministic phases: **Markdown Normalization**, **HTML & CSS Template Injection**, **Headless Chromium Execution**, and **Antigravity Artifact Synchronization**.

Component	Responsibility	Technology Stack
Frontmatter & Markdown Parser	Parse metadata, normalize GitHub callout badges, extract headings	`python-frontmatter`, `markdown`
Math & Diagram Engine	Vector math formula layout, Mermaid diagram rendering	KaTeX, Mermaid.js
Theme System	Curated styling (`modern` , `academic` , `report` , `minimal` , `github`)	CSS Paged Media (`@page`), Flexbox
Browser Runner	Headless Chrome/Edge execution with compositor stage stabilization	Headless Chromium, Virtual Time Budget
Artifact Synchronizer	Generative UI card and Markdown summary generation	Antigravity Design System, TailwindCSS

> 💡 **TIP**
>
> The engine requires zero external C-dependencies or GTK/Cairo runtimes. On Windows, it leverages the pre-installed Google Chrome or Microsoft Edge browser natively, ensuring high speed and complete portability.

---

3. Mathematical Formula Typesetting

The system provides first-class support for both inline math expressions, such as $E = mc^2$ and $e^{i\pi} + 1 = 0$, and multi-line display equations.

For example, the classic Gaussian integral formulation:

$$\int_{-\infty}^{+\infty} e^{-x^2}\,dx = \sqrt{\pi}$$

As well as regularized loss functions commonly used in deep learning:

$$\mathcal{L}_{\text{total}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \left[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right] + \frac{\lambda}{2}\|\mathbf{W}\|_2^2$$

> 📌 **IMPORTANT**
>
> The headless browser runner allocates a `--virtual-time-budget=3000` execution window. This guarantees that all client-side KaTeX fonts and SVG layouts are fully evaluated before capturing the PDF print stream.

---

4. Code Block Syntax Highlighting

Technical code snippets are styled with Pygments syntax highlighting, paired with clean monospace typography (`Cascadia Code`, `JetBrains Mono`, `Consolas`):

```python
from pathlib import Path
from scripts.md_to_pdf import convert_markdown_to_pdf

def generate_technical_brief():
    result = convert_markdown_to_pdf(
        input_file=Path("architecture_spec.md"),
        output_pdf=Path("dist/architecture_spec.pdf"),
        theme="report",
        cover_page=True,
        enable_toc=True,
        artifact_dir=Path(".artifacts"),
    )
    print(f"Generated {result['page_count']} pages | Size: {result['file_size']}")

if __name__ == "__main__":
    generate_technical_brief()
```

---

5. Core Feature Matrix

- [x] Full support for 5 GitHub-style Callouts (`NOTE`, `TIP`, `IMPORTANT`, `WARNING`, `CAUTION`)
- [x] High-precision LaTeX inline and block math formula rendering
- [x] Multi-theme visual layouts (`modern`, `academic`, `report`, `minimal`, `github`)
- [x] Formal cover page layout with author, date, version, and abstract metadata
- [x] Automatic Table of Contents (`--toc`) generation
- [x] Bi-directional Antigravity Generative UI Artifact synchronization
- [ ] Planned: Dynamic watermark and custom page background overlays

> ⚠️ **WARNING**
>
> When compiling long-form documents containing external web assets, ensure stable network connectivity so that remote CDN resources resolve properly during the print phase.

---

6. Conclusion

With `markdown-to-pdf`, Antigravity seamlessly elevates raw markdown drafts into polished, publication-ready PDF deliverables, empowering developers and researchers with instantaneous report generation.